☑ What is an Array?

An **array** is a collection of elements (numbers, characters, etc.) stored under a single variable name.
Each element is accessed using an **index**.

Example:

```
Marks = [10, 20, 30, 40]
Index   0   1   2   3
```

☑ Part 1: 1D Array (One-Dimensional Array)

A **1D array** stores elements in a single row or list.

Syntax in Different Languages

Language	Syntax
Python	`arr = [10, 20, 30]`
C++	`int arr[3] = {10, 20, 30};`
Java	`int[] arr = {10, 20, 30};`

---

1. LeetCode 1480 — Running Sum of 1D Array

Compute cumulative sum.

Python

```
def runningSum(nums):
    for i in range(1, len(nums)):
        nums[i] += nums[i - 1]
    return nums
```

C++

```
vector<int> runningSum(vector<int>& nums) {
    for(int i = 1; i < nums.size(); i++)
        nums[i] += nums[i - 1];
    return nums;
}
```

Java

```
int[] runningSum(int[] nums) {
```

```
    for (int i = 1; i < nums.length; i++)
        nums[i] += nums[i - 1];
    return nums;
}
```

---

2. LeetCode 1929 — Concatenation of Array

Python

```
def getConcatenation(nums):
    return nums + nums
```

C++

```
vector<int> getConcatenation(vector<int>& nums) {
    vector<int> res = nums;
    res.insert(res.end(), nums.begin(), nums.end());
    return res;
}
```

Java

```
int[] getConcatenation(int[] nums) {
    int[] res = new int[nums.length * 2];
    for(int i = 0; i < nums.length; i++) {
        res[i] = nums[i];
        res[i + nums.length] = nums[i];
    }
    return res;
}
```

---

3. LeetCode 1672 — Richest Customer Wealth

Python

```
def maximumWealth(accounts):
    return max(map(sum, accounts))
```

C++

```
int maximumWealth(vector<vector<int>>& accounts) {
    int maxWealth = 0;
    for(auto& acc : accounts) {
        int sum = 0;
        for(int m : acc) sum += m;
        maxWealth = max(maxWealth, sum);
    }
    return maxWealth;
}
```

Java

```
int maximumWealth(int[][] accounts) {
    int max = 0;
    for(int[] acc : accounts) {
        int sum = 0;
        for(int money : acc) sum += money;
        max = Math.max(max, sum);
    }
    return max;
}
```

---

4. LeetCode 1431 — Kids With Greatest Candies

Python

```
def kidsWithCandies(candies, extra):
    max_c = max(candies)
    return [c + extra >= max_c for c in candies]
```

C++

```
vector<bool> kidsWithCandies(vector<int>& candies, int extra) {
    int max_c = *max_element(candies.begin(), candies.end());
    vector<bool> res;
    for(int c : candies)
        res.push_back(c + extra >= max_c);
    return res;
}
```

Java

```
List<Boolean> kidsWithCandies(int[] candies, int extra) {
    int max = 0;
    for (int c : candies) max = Math.max(max, c);

    List<Boolean> res = new ArrayList<>();
    for (int c : candies)
        res.add(c + extra >= max);
    return res;
}
```

---

5. LeetCode 1365 — Smaller Numbers Than Current

Python

```
def smallerNumbersThanCurrent(nums):
    return [sum(n < x for n in nums) for x in nums]
```

C++

```
vector<int> smallerNumbersThanCurrent(vector<int>& nums) {
    vector<int> res;
    for(int i = 0; i < nums.size(); i++) {
```

```
        int count = 0;
        for(int j = 0; j < nums.size(); j++)
            if(nums[j] < nums[i]) count++;
        res.push_back(count);
    }
    return res;
}
```

Java

```
int[] smallerNumbersThanCurrent(int[] nums) {
    int[] res = new int[nums.length];
    for(int i = 0; i < nums.length; i++) {
        int count = 0;
        for(int j = 0; j < nums.length; j++)
            if(nums[j] < nums[i]) count++;
        res[i] = count;
    }
    return res;
}
```

LeetCode 560 — Subarray Sum Equals K (O(n²))

Logic: Try all possible subarrays and count those whose sum equals `k`.

☑ Python

```
def subarraySum(nums, k):
    n = len(nums)
    count = 0

    for i in range(n):
        total = 0
        for j in range(i, n):
            total += nums[j]
            if total == k:
                count += 1

    return count
```

☑ C++

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    int subarraySum(vector<int>& nums, int k) {
        int n = nums.size();
        int count = 0;

        for (int i = 0; i < n; i++) {
```

```
            int sum = 0;
            for (int j = i; j < n; j++) {
                sum += nums[j];
                if (sum == k)
                    count++;
            }
        }
        return count;
    }
};
```

☑ Java

```
class Solution {
    public int subarraySum(int[] nums, int k) {
        int n = nums.length;
        int count = 0;

        for (int i = 0; i < n; i++) {
            int sum = 0;
            for (int j = i; j < n; j++) {
                sum += nums[j];
                if (sum == k)
                    count++;
            }
        }
        return count;
    }
}
```

☑ Part 2: 2D Array (Matrix)

A 2D array is an **array of arrays**, like a table.

Example:

```
[
 [1, 2, 3],
 [4, 5, 6]
]
```

Syntax

Language	Syntax
Python	`arr = [[1,2],[3,4]]`
C++	`int arr[2][2] = {{1,2},{3,4}};`
Java	`int[][] arr = {{1,2},{3,4}};`

Common Operations

Print Matrix

```
for row in arr:
    for val in row:
        print(val, end=" ")
    print()
```

Sum of Elements

```
total = sum(sum(row) for row in arr)
```

Transpose

```
for i in range(n):
    for j in range(m):
        print(arr[j][i], end=" ")
```

Maximum Element

```
max_val = max(max(row) for row in arr)
```

Count Even & Odd

```
even = odd = 0
for row in arr:
    for val in row:
        if val % 2 == 0:
            even += 1
        else:
            odd += 1
```

☑ Dynamic Arrays

C++ Vector

```
vector<int> v;
v.push_back(10);
v.pop_back();
cout << v.size();
```

Java ArrayList

```
ArrayList<Integer> arr = new ArrayList<>();
arr.add(10);
arr.remove(arr.size()-1);
```

Python List

```
arr = []
arr.append(10)
arr.pop()
```